Robust Malware Detection using Integrated Static and Dynamic Analysis

A thesis submitted in fulfillment of the requirements

for the degree of Master of Technology

by

Pranjul Ahuja

Department of Computer Science And Engineering

INDIAN INSTITUTE OF TECHNOLOGY KANPUR

July 2017

Statement of Thesis Preparation

1. Thesis title: Robust Malware Detection using Integrated Static and Dynamic Analysis.
2. Degree for which the thesis is submitted: M.Tech
3. Thesis Guide was referred to for preparing the thesis.
4. Specifications regarding thesis format have been closely followed.
5. The contents of the thesis have been organized based on the guidelines.
6. The thesis has been prepared without resorting to plagiarism.
7. All sources used have been cited appropriately.
8. The thesis has not been submitted elsewhere for a degree.

P. Ahuja
(Signature of the student)

Name: Pranjul Ahuja

Roll No.: 15111029

Department/IDP: CSE

POST GRADUATE OFFICE
THESIS SUBMITTED
ON 10/7/17
I. I. T., KANPUR - 16

Certificate

It is certified that the work contained in this thesis entitled "Robust Malware Detection using Integrated Static and Dynamic Analysis" by "Pranjul Ahuja" has been carried out under my supervision and that it has not been submitted elsewhere for a degree.

10.7.17

Dr. Sandeep Shukla

Professor

Department of Computer Science and Engineering

Indian Institute of Technology Kanpur

July 2017

Abstract

Name of the student: **Pranjul Ahuja** Roll No: **15111029**

Degree for which submitted: **M.Tech.** Department: **Computer Science and Engineering**

Thesis title: **Robust Malware Detection using Integrated Static and Dynamic Analysis**

Thesis supervisor: **Dr. Sandeep Shukla**

Month and year of thesis submission: **July 2017**

There has been an exponential growth in the number of malware in the cyber world in the last few years. Modern malwares use sophisticated techniques such as polymorphism and metamorphism to thwart the malware detection and analysis. Detecting malware on the basis of their features and behavior is critical for the computer security community. Most anti-virus depends on the signature based detection which is relatively easy to evade and is ineffective for zero-day exploit based malwares. Static analysis analyzes the executables without executing them whereas dynamic analysis actually executes the malware in a sand-boxed environment and the system changes are logged for further investigation. In this thesis, we are adopting a hybrid approach in which we integrate the feature vectors extracted from both static and dynamic analysis to detect unknown malware. Our experiments obtained an accuracy of 98.62% in detecting malware. Our detection system is robust and scalable as we have increased the amount of samples used for analysis and reduced the feature space compared to the existing approaches in the literature.

Acknowledgements

I would extend my sincerest gratitude to Dr. Sandeep Shukla for guiding me in this thesis. I would also like to thanks Vineet Purswani and Ajay Chaudhary for their cooperation during various phases of the project. I am very grateful to VirusTotal Community for sharing their private API to help us build our dataset. I am thankful to my parents and sister for the love they have given me. Lastly, I would like to thank all of my lab mates including Mugdha Gupta, Avik Dayal, Neha Ajmani and Abhay Kumar for their motivational support.

Contents

Abstract iii

Acknowledgements iv

Contents v

List of Figures vii

List of Tables viii

1 Introduction 1
- 1.1 Need For Malware Detection . . . 1
- 1.2 Goals of the Project . . . 2

2 Problem Isolation 4
- 2.1 Malware Detection . . . 4
  - 2.1.1 Signature Based (Static Analysis) . . . 5
  - 2.1.2 Behavior Based (Dynamic Analysis) . . . 6
- 2.2 The Motivation for The Hybrid Approach . . . 7
- 2.3 Contribution of the Thesis . . . 7

3 Past Work 9
- 3.1 Static Analysis Based Feature Extraction . . . 9
- 3.2 Dynamic Analysis Based Feature Extraction . . . 10
- 3.3 Hybrid Analysis Based Feature Extraction . . . 11

4 Feature Extraction and Integration 13
- 4.1 Analysis and Detection System . . . 13
  - 4.1.1 Architecture . . . 13
    - 4.1.1.1 Data Generation . . . 13
    - 4.1.1.2 Static Feature Extraction . . . 17
      - 4.1.1.2.1 PE Header Feature Vectors . . . 17
      - 4.1.1.2.2 Dealing with obfuscation : Entropy Feature Vectors . . . 21
      - 4.1.1.2.3 String Based Features . . . 22

4.1.1.3 Dynamic Feature Extraction 23
4.1.1.3.1 File System Changes Related Features 24
4.1.1.3.2 Windows Registry Based Features 27
4.1.1.3.3 Mutex Based Features 28
4.2 Machine Learning Classifiers: **Scikit-Learn** 30
4.3 Summary 32

5 Experimental Results **34**
5.1 Malware and Benignware Dataset 34
5.2 Evaluation Metric 34
5.2.1 Confusion Matrix 35
5.2.2 Additional Metrics 35
5.3 Training and Testing 36
5.3.1 The entire description of K-fold Cross-validation 36
5.4 Comparison To Existing Approaches 39

6 Scope and Future Work **41**
6.1 Multi-Path Execution in Cuckoo sandbox 41
6.2 Performing Classification of malware families 41
6.3 Providing Malware detection and classification as a cloud based service 42

A Appendix A **43**

Bibliography **55**

List of Figures

1.1 Growth of malware 1

4.1 Architecture of our Integrated Method 14
4.2 Microsoft PE Format 15
4.3 Cuckoo Architecture[4] 16
4.4 Codes For different CPU's[9] 17
4.5 Bit Flags [9] 18
4.6 Data Directory Entires[9] 19
4.7 Distribution Of Number Of Sections 20
4.8 Distribution Of SizeOfHeaders Field 21
4.9 String command output on some executable 23
4.10 Cuckoo Analysis Report in JSON Format 24
4.11 Frequent Directories Touched by malware 25
4.12 Frequent Directories Touched by benign-ware 26
4.13 Extensions Touched by malware 27
4.14 Extensions Touched by benign-ware 27
4.15 Legitimate Mutexes 29
4.16 Non-legitimate Mutexes 29

5.1 Binary Confusion Matrix 35
5.2 K-Fold Cross Validation 37
5.3 Confusion Matrix For XG-Boost on integrated Feature Set 39

List of Tables

3.1 Mean value comparison for Various attributes in PE Headers 10
3.2 Mean value comparison for Various attributes in PE Headers 11

4.1 Mean value comparison for Various attributes in PE Headers 21

5.1 Test Results with static features set (PE Headers + Entropy + String Histograms) . . 38
5.2 Test Results with integrated feature set 38
5.3 Past work done in Hybrid Analysis 40

Dedicated to my parents...

Chapter 1

Introduction

1.1 Need For Malware Detection

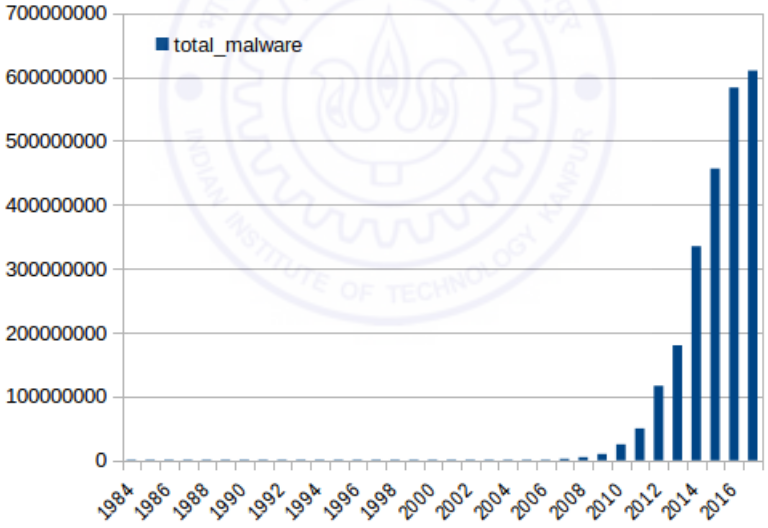

FIGURE 1.1: Growth of malware

Over the past few years, The Internet has become a vital part in our lives with increased usage of services like online banking, online reservation etc., and our dependence on the Internet is expected to grow. With the rise of the Internet, there has been huge growth in the amount of malware samples in the wild as depicted in the Figure 1.1. It is very concerning that despite the advancements in the anti-virus technologies, malware authors keep on coming up with more sophisticated approaches to evade these anti-virus and anti-malware protection. According to the report by AV-TEST institute[2], 390,000 new malicious programs are registered everyday on

their website.
Analyzing such huge amount of malware manually using techniques such as reverse engineering, unpacking and disassembly would require a lot of human effort and seems to be infeasible for such growth. This has been the motivation for the malware analysts to build automated system for detecting malicious software. Mostly two analysis techniques are used to build such kind of detection system : static and dynamic. Static analysis aims to analyze the sample without executing it and it is generally fast. However, it does not give useful insights in case the sample is packed, encrypted or obfuscated. As a result, attention has moved on to the dynamic analysis in which a malware is executed in a sandbox and all the activities are monitored. While dynamic analysis may not be thwarted by obfuscation, it also has got some drawbacks. There are many sophisticated malware which first detects if they are being analyzed in a sandbox and then decide whether to launch their malicious payload. It is also difficult to get a complete picture of a malware since dynamic analysis does not explore all possible execution paths inside a sandbox.
It is therefore not reliable to depend on a single analysis to build an effective and robust malware detection system. The main issue in the hybrid approach though, is the size of the feature space which might incur scalability issues in case we have to analyze a large amount of malware samples. In this thesis, we develop a hybrid malware detection system with precision comparable to the most effective systems reported in the literature and with added scalability.

1.2 Goals of the Project

Most of the work in malware detection have focused on either static analysis or dynamic analysis. One more problem is the number of samples used in the learning process. We can build more robust detection system by building models on large dataset. Some other problems include unpacking the malware initially before starting the analysis which can waste a lot of time. The main goals of the project is to improve the system by :

- Integrating the static and dynamic analysis feature vectors for better robustness.
- Using entropy based features to deal with packed malware.
- Reducing the feature space as compared to the existing approaches and hence, achieve scalability.
- Using machine learning as a data mining and prediction tool for efficient detection of malware.

- Building predictive models on large datasets of malware and cleanware for robust classification.

Chapter 2

Problem Isolation

2.1 Malware Detection

The term **Malware** is used when referring to a malicious software, which is designed to disrupt or gain unauthorized access to a system. There are various types of malware, brief description of each is given below :

- **Viruses and Worms** : These are famous for the way they spread to other systems. Virus can infect any normal file or program on the system by attaching itself to it and will infect the whole system as well when executed. On the other hand, worms spreads automatically using the computer network and performs various malicious actions.
- **Spy-Wares** : These programs are crafted to intrude on the privacy of users by leaking confidential information like screen-shots, on-line banking credentials or Internet surfing history etc of users to the third party without user's knowledge. They get installed on the system very secretly and could be difficult to detect.
- **Trojans** : Often referred as Trojan Horses, are programs designed to have an appearance of serving a useful utility but actually contains some hidden malicious payload which may be to install spy-wares.
- **Root-kits** : These are collection of tools which are used to gain administrator/root privileges on a system by exploiting a known vulnerability in the kernel or through passwords (Brute-forcing commonly used passwords or using social engineering). Once installed, they are very difficult to detect.

- **Backdoors** : These are used by attackers to gain unauthorized remote access to a system using undocumented ways bypassing security policies.
- **RansomWare** : It is a type of malware which inhibits the access to a system by either locking the screen or rendering the file system useless by encrypting it. They will ask for ransom in exchange for giving back the access. There is no guarantee that a user will get access even after paying huge ransom.

It is therefore required to differentiate between a malware and a cleanware once an unknown file has entered into the system in order to ensure it will not perform any malicious activity. Several ways by which a malware could enter in your system include :

- Downloading a file from Internet.
- Opening attachments in email or social networking websites such as face-book.
- Plugging external hard drives, USB and CD's to the system.
- Browsing infected web-pages.

This thesis aims to perform malware detection, where models will be trained on large corpus of executables using good set of discriminative predictors extracted through static and dynamic analysis. Our system has the ability to detect zero-day malwares compared to normal anti-virus software which detect by just matching the hashes computed from the malware source code or binary in their database of already registered malware. Zero day malware are those which has not been detected yet.

2.1.1 Signature Based (Static Analysis)

Static analysis is the technique of getting insights into the malware functionalities by examining its static properties. Without executing the malware, we can examine various properties such as the strings embedded in the binary, PE Header, embedded resource directories, meta-data and disassembly etc. Various techniques which are being used by malware authors to thwart the static analysis are briefly described below :

- **Packing** : It is a technique of applying compression or encryption algorithms to the original executable and transform it into a packed executable which contains a unpacking stub and original packed code. When this packed binary is executed, OS loader will first

load the unpacking stub which will unpack the application in the memory, bring the necessary imports, exports and all the sections and then transfer the control to the original entry point of the executable.

- **Metamorphism** : Metamorphic malware consist of a code mutation engine which will change the code in every execution but the algorithm/functionality remains the same. It is generally achieved through changing registers, replacing instructions with equivalent ones with different operands, garbage code insertion and reordering the sub-routines within the program etc. This will make the signature of malware different and hence difficult for normal anti-virus software to detect.

- **Polymorphism** : This term is generally used for decryption engine present in the malwares. Decryptors would contain variable elements which are generally fixed size instructions and are used to affect the size or shape of the code. Polymorphic malwares contains potentially highly variables elements which can allow changes like subroutine creation, algorithmic register initialization etc., and making it difficult to capture the signature of unpacking stub. Consider the following instruction in the decryptor

$$mov\ ax,\ ax;$$

  It can be easily replaced with registers bx or cx and we will get a new decryption routine with new signature.

There are various indicators which makes it difficult to statically analyze an obfuscated/packed samples such as :

- Low number of imports in the PE Header of executable. LoadLibrary and GetProcAddress are generally used to load the required API at runtime.
- After disassembling the binary, only small portion of the code would be recognizable.
- After packing the binary, the string utility will only show the strings of the unpacking stub and not of the original executable.

2.1.2 Behavior Based (Dynamic Analysis)

In order to avoid the various shortcoming of static analysis, attention of malware analysts has moved on to dynamic analysis based methods. These methods generally involves running the

executable in a confined environment providing a bare minimum of resources for execution such as the Cuckoo Sandbox and then analyzing the post-execution behavior reports.
The main advantage is not being affected by the polymorphic or metamorphic techniques since these methods are completely independent of the source code of the malware. Some of the disadvantages of these methods includes time and resource intensive analysis compared to static counterpart, are prone to analysis evasion as some malware avoid launching their malicious payload after sensing the sand-boxed/virtualized environment and are not able to explore all the execution paths of an executable giving a partial picture.

2.2 The Motivation for The Hybrid Approach

Both the analysis techniques are complementary to each other. Dynamic analysis can give useful insights even in case of obfuscated malware when static won't be able to provide sufficient information. On the other hand, static analysis can give a complete overview of an executable when dynamic analysis is subjected to less code coverage. This led us to believe that a single component can be made incorporating the features from both the worlds and the results were highly promising. Anti-virus solutions analyzing an executables based on PE headers based features can be thwarted by adding redundant values to various header fields which might not match with the specifications of header. Anti-virus solutions based on string based features can be thwarted by adding superfluous strings in the executable. Run time analysis can be avoided by adding useless functionalities in the malware such as accessing some random files, creating random registry values or holding some legitimate mutexes unnecessarily etc. This integrated component would make it very difficult for malware writers to add changes to all the components simultaneously without making the malware useless or very easy to detect.

2.3 Contribution of the Thesis

The major contribution of this thesis include :

- We developed a robust malware detection system combining static and dynamic analysis while considerably reducing the feature set. Further, in our work, we have not manually unpacked/de-obfuscate any of the samples which takes a lot of time as we have used entropy based features for detecting packed malwares.
- We evaluated our models on large corpus of malwares downloaded for virustotal[10] and compare performance with existing work to show effectiveness.

- We have released all the code used in the analysis on github:
  https://github.com/ahuja0007/Automated-Malware-Analysis

Chapter 3

Past Work

3.1 Static Analysis Based Feature Extraction

Conventional classification methods have been relying on static feature extraction using reverse engineering, examining DLL usage from PE Headers and Strings, [20] used list of functions inside DLL called by binary , string features and byte sequences using hex-dump to perform classification. [12] also used sequences of API called under windows, extracted using decompilation analysis. Another technique known as the n-gram sequence of byte code obtained after applying the hex-dump utility has been used for higher accuracy by [16]. Another frequently used approach is based on extracting the strings [19] from the executable using 'strings' utility from Linux. Authors of [27] used IDA to extract strings and perform classification. Though strings feature has been successful in classification with greater accuracy, they are not helpful in case the malware is obfuscated. Packers usually render almost all the strings inside the executable unprintable or just random characters. Since there is a lot of gibberish along with actual string data, we have not used them directly. We have only used frequency bins related to the distribution of strings inside the executable.

Another interesting approaches used in the classification process are op-code sequences [15] and function lengths [24]. Operation codes (Opcode) are basically mnemonic representation of machine code and various techniques such as frequency distribution, cosine similarity[26], critical instruction sequences have been used in the detection process. Function analysis is done by segmenting the binary into functions by disassemblers like IDA [5] and then using frequency of function lengths which is the number of bytes as defined by the disassembler. However, these approaches are not robust and feasible since some of the executables can not be disassembled easily.

Table 3.1 shows the accuracy scores using different feature representations based on dynamic analysis. Ineffectiveness of the static analysis based feature extraction methods against packed/obfuscated malwares shifted the focus of malware analysts towards run time feature extraction methods.

No.	Feature Used	Accuracy
1	API calls sequence	93.71%
2	n-gram on byte codes	98%
3	Printable string information	97%
3	Function length using IDA	87.76%

TABLE 3.1: Mean value comparison for Various attributes in PE Headers

3.2 Dynamic Analysis Based Feature Extraction

Dynamic analysis aims to gain useful insights by monitoring the run time behavior of executable. [29] used the Win32 API run time calls by hooking the API functions imported in the executable using Detours technology developed by Microsoft. Another interesting approach was using behavior graphs [22] constructed based on the dependencies of the system calls executed by executables in virtualized environment and then classifying using maximal common sub-graph detection. In [17], authors targeted their focus on root-kits which uses stealthy and obfuscated ways to evade detection and used McAfee's Rootkit Detective to get all the in line function hooks used by root-kits. Authors of [31] have used a novel technique of applying DNA sequence alignment algorithms to the runtime API sequence revealed by malwares and extracted some common API sequence patterns found in different types of malwares.

Fine grained approaches used for dynamically analyzing the malwares are helpful in getting detailed information for the executed binaries but they generally come with the cost of high dimensional feature vectors which might incur high performances penalties. [28] is an example of one such fine grained system in which authors created behavior profiles of malware by integrating taint and network analysis into the QEMU based sandbox ANUBIS and then clustered the malware using hierarchical clustering algorithm. On the other hand, coarse grained analysis has been effective in characterizing malicious software and in improving scalability by reducing the feature space as well as noise in the extracted features. [18] is one such coarse grained dynamic analysis system which proposed a scalable malware modeling technique using a test engine which implements a test case such as editing a document, visiting a website etc and a taint engine tainting nine different sources in system such as text, document, HTTP, HTTPS, FTP, Directory and password.

In all these above mentioned works, malicious samples are executed in controlled environment using a sandbox inside a virtual machine. Though it is artificial and many malware authors have been coming up with malware which can sense the virtualized environments and therefore evade the analysis, it is still most commonly used in behavior analysis enabling analysts to rapidly analyze large corpus of executables and restore clean environment after the examination of each sample. Inability to explore all possible execution paths of an executable still remains one of the most important problem in this area. Table 3.2 shows the accuracy scores using different feature representations based on dynamic analysis. Some of the authors have also done clustering of malware apart from classification.

No.	Feature Used	Accuracy
1	Frequency of API usage	98.50%
2	Run time API call sequences	99.8%
3	Printable string information	97%

TABLE 3.2: Mean value comparison for Various attributes in PE Headers

3.3 Hybrid Analysis Based Feature Extraction

Considering the limitations of both the approaches, it is very risky to build a malware detection/classification system relying purely either on static or dynamic analysis. Therefore recently, malware analysts have started digging in the area of combining the extracted features from both the analysis to perform more robust detection and classification.

Authors of [14] have used the frequency of opcode sequences for their static component and logged relevant Windows API for their dynamic part on a dataset of around 1000 malware and 1000 benign samples to achieve detection accuracy of 96%.

In [23], authors have combined frequency bins based on the length of functions obtained by disassembling the binaries, Printable string information from static analysis and Win32 API calls from dynamic analysis on a dataset of 2300 malwares and 541 benign files achieving accuracy of 97%. All the packed malware samples were first unpacked before extracting the features.

Authors of [13] proposed a integrated framework in which they used the sizes and count of various sections present in the PE file, frequency of Wind32 functions retrieved from PE Header and network related features to distinguish malwares from clean-wares. 998 malicious and 428 benign files were used to achieve an accuracy of 99%.

The major problems in these approaches is that feature space grows in proportion with the number of files because of the direct usage of strings and Win32 API as binary features. One more concern is the number of samples used in the detection process. In our thesis, we have

tried building models on a comparatively large dataset and low dimensional feature space and achieved promising results.

Chapter 4

Feature Extraction and Integration

4.1 Analysis and Detection System

In this section, we will describe the architecture of our detection system in which we have incorporated various feature engineering and modeling techniques to achieve good detection accuracy. We will also go through the architecture of our dynamic analysis setup in which we have used open-source sandbox Cuckoo for analyzing run time behavior of executables inside virtual machine.

4.1.1 Architecture

The architecture of our system is described in Figure 4.1. In this work, we are only focusing on analyzing 32 bit PE windows executables but our work can be extended to 64 bit versions as well. Our system consists of 3 parts namely : Data Generation, Feature Extraction and Data Modeling.

4.1.1.1 Data Generation

The modules included in data generation phases are responsible for performing static and dynamic analysis on the corpus of executables and storing the post-execution reports on the system for the next phase. These include :

- **GNU Strings** : Strings is one of important utility present in the collection of GNU Binutils. When executed on any file, it will search for the sequence of three or more printable

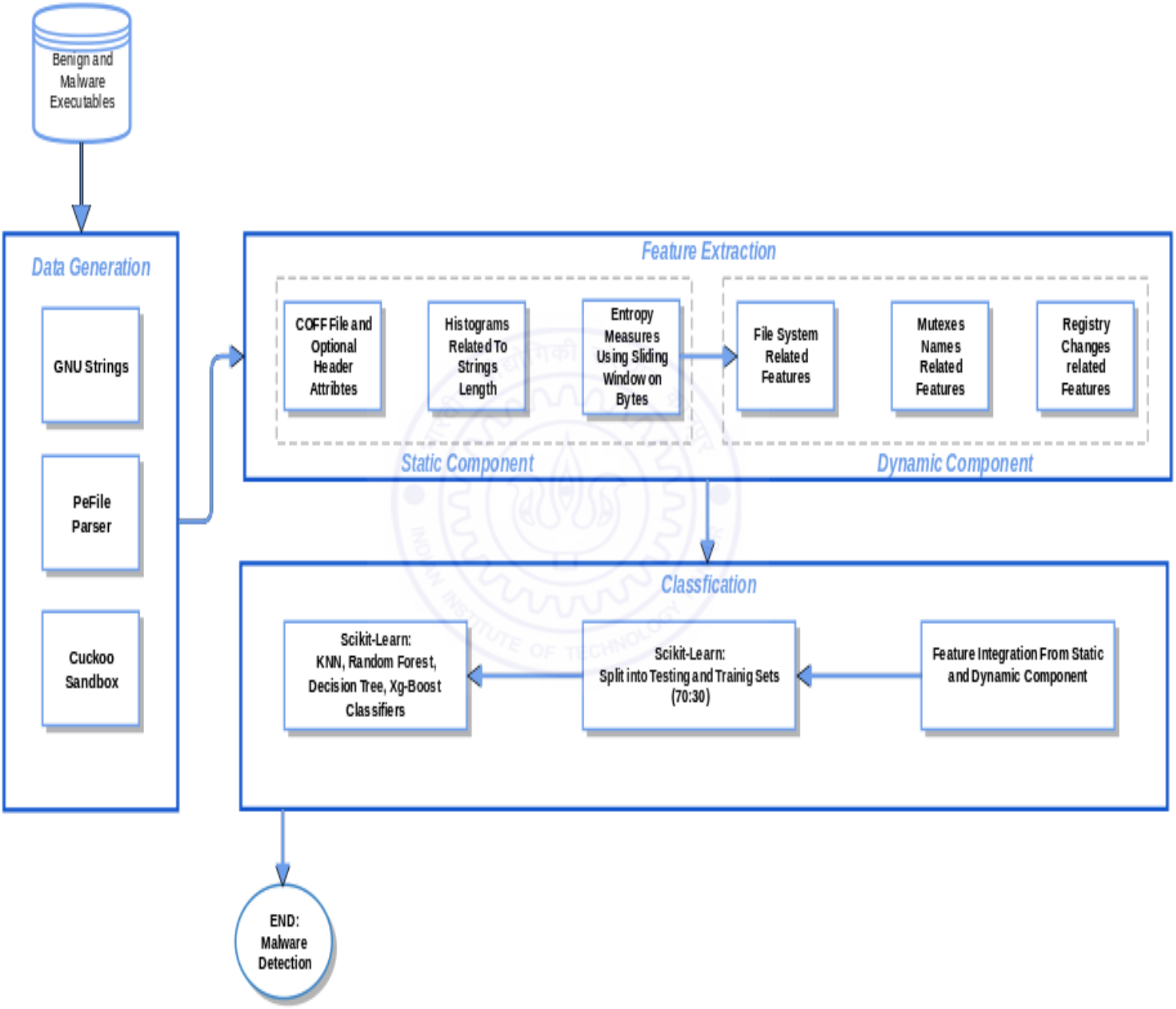

FIGURE 4.1: Architecture of our Integrated Method

ASCII characters followed by an unprintable/termination character. Analyzing the output of strings on any executables can provide many hints what an executable might be trying to do on a system. For example, if a malware tries to open a UDP connection to its author for command and control, we will see some IP addresses in the output of strings present inside the malware.

- **Python PeFile** : *pefile* is a multi-platform Python module to parse and work with Portable Executable(PE) files which is a standard binary format for a DLL or an executable in Microsoft Windows. PE file format contains plethora of useful information in the form of multiple headers arranged in the format shown in Figure 4.2.

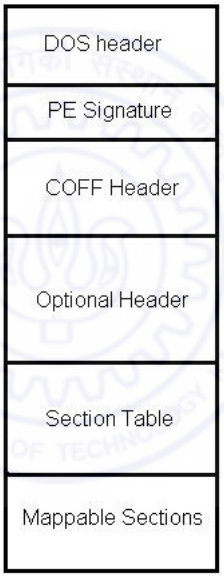

FIGURE 4.2: Microsoft PE Format

For detection, we have used only COFF and optional header. COFF Header contains useful information such as NumberOfSections, SizeOfOptionalHeader etc. The optional header appears directly after the COFF header, which includes rich information about executable such as SizeOfCode, SizeOfHeaders and many other important features about the binary.

- **Cuckoo Sandbox** : Cuckoo is a tool which allows us to perform sandboxed malware analysis by creating an isolated environment and giving us the information on what it does

inside. The main components of Cuckoo's sandbox include a host machine which manages all the analysis process and number of guest machines which are responsible for executing the samples. We are using latest Ubuntu 16.04 on host machine with 16 GB RAM and 3 virtual machines each containing 32 bit Windows 7 with no updates or Anti-virus program. The User Access Control (UAC) is disabled in all the VMs in order to maximize the activities of a malware.

In order to analyze a sample, analyst need to add its path in the database of cuckoo which is maintained in the host machine. Each analysis is launched in a fresh virtual machine. While setting up the virtual machines, snapshot of current state is taken so that Cuckoo can revert back after the execution of malware which generally takes 2-3 seconds. We are using the default setting of 120 seconds for which malware sample will be monitored. After this period, it is terminated by Cuckoo. All the changes made by the malwares on the system are recorded and stored in the JavaScript Object Notation (JSON) format on the host machine. Main architecture is described in the following Figure 4.3 :

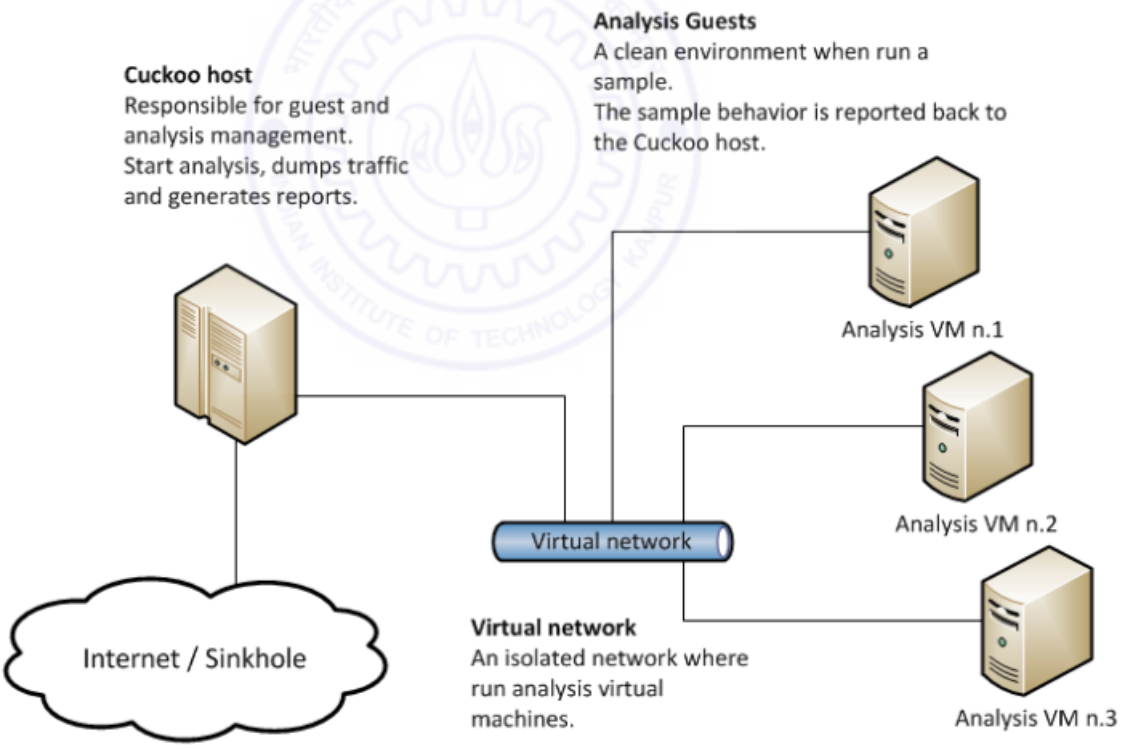

FIGURE 4.3: Cuckoo Architecture[4]

4.1.1.2 Static Feature Extraction

In this section, we will explain some of the features extracted from the executables using only static analysis.

4.1.1.2.1 PE Header Feature Vectors

The PE File Header[9] in Windows executables generally consists of following :

- **PE signature** : It is a string ('PE\0\0') placed after the MS-DOS stub where each '\0' is a ASCII Null character. This signature is used to show that the file is a legitimate PE file and also the byte order of the file. All the PE files are generally in "little endian" format.
- **COFF File Header** : This header is present both in executable as well as object files. It has information useful for both object as well as PE executables. It includes the following :
  - **Machine** : It is a 16 Bit field which is used to represent a machine the binary is compiled for. Figure4.4 shows the possible values it can have.

Value	Description
0x14c	Intel 386
0x8664	x64
0x162	MIPS R3000
0x168	MIPS R10000
0x169	MIPS little endian WCI v2
0x183	old Alpha AXP
0x184	Alpha AXP
0x1a2	Hitachi SH3
0x1a3	Hitachi SH3 DSP
0x1a6	Hitachi SH4
0x1a8	Hitachi SH5
0x1c0	ARM little endian

FIGURE 4.4: Codes For different CPU's[9]

  - **NumberOfSections** : 16 bit field used for the count of sections in the file after the PE headers. There are some common sections which can be found in both EXE and OBJ files.
    .txt section contains all the code emitted by compilers. *.data* is the section where all the initialized data is stored. All the global, static variables as well as string literals

are present in data section. All the uninitialized global and static variables are stored in *.bss sections.*

- **TimeDateStamp** : 32 bit field holds the number of seconds since December 31st, 1969 to indicate the time, the linker produced this file.
- **SizeOfOptionalHeader** : This is the size of "PE Optional Header" which will follow the COFF header. For OBJ files, this field is 0. For EXE files, it includes the size of IMAGE_OPTIONAL_HEADER that will follow after this header.
- **Characteristics** : It is a bit flag containing information about file. Some important fields are presented in Figure4.5.

0x0001	There are no relocations in this file
0x0002	File is an executable image (not a OBJ or LIB)
0x2000	File is a dynamic-link library, not a program

FIGURE 4.5: Bit Flags [9]

- **NumberOfSymbols** : The number of symbols in COFF symbol table which is used by both OBJ and EXE files for storing debug information.

• **Optional Header** : It is variable in size and contains multiple sub-fields described below :

- **Standard Fields** : Magic number which determines whether executable is 64 bit (PE32+) or 32 bit (PE32), size of the code for cumulative size of all the code sections used in bytes, linker version and address entry point which is a pointer to the entry point function for the executable.
- **Windows Specific Fields** : Target OS, SizeOfHeaders which include 4 byte PE signature, COFF header, Optional header and all the sections headers , ImageBase which holds the first byte of the image when loaded into memory, SizeOFImage includes size of the image along with the headers in bytes, file alignment used for the alignment of data of sections in image in bytes, section alignment used for the alignment of sections loaded in memory in bytes in image in bytes and number of data directory entries present in the remainder of optional header.

– **Data Directories** : An array of IMAGE_DATA_DIRECTORY structures where each entry consists of the address and the size of a table required for execution such as import table, export table and resource table. Figure 4.6 shows all the entries present in this field.

There may be lot of malformations which can be introduced by malware writers to the various fields described above inside PE Header, in case they are not aware of the PE/COFF specifications. But while executing the files, window loader has to maintain backward compatibility with the obsolete compilers and therefore behaves fault tolerant while facing invalid structures inside headers.

We used *pefile* python module to scrape every minute detail from the PE headers of the executables. Figure 4.7 and Figure 4.8 are some examples of data visualizations which we have used to see the distribution of values of the PE Header Fields.

Constant Name	Value	Description	Offset PE(32 bit)
IMAGE_DIRECTORY_ENTRY_EXPORT	0	Export Directory	96
IMAGE_DIRECTORY_ENTRY_IMPORT	1	Import Directory	104
IMAGE_DIRECTORY_ENTRY_RESOURCE	2	Resource Directory	112
IMAGE_DIRECTORY_ENTRY_EXCEPTION	3	Exception Directory	120
IMAGE_DIRECTORY_ENTRY_SECURITY	4	Security Directory	128
IMAGE_DIRECTORY_ENTRY_BASERELOC	5	Base Relocation Table	136
IMAGE_DIRECTORY_ENTRY_DEBUG	6	Debug Directory	144
IMAGE_DIRECTORY_ENTRY_ARCHITECTURE	7	Architecture specific data	152
IMAGE_DIRECTORY_ENTRY_GLOBALPTR	8	Global pointer register relative virtual address	160
IMAGE_DIRECTORY_ENTRY_TLS	9	Thread Local Storage directory	168
IMAGE_DIRECTORY_ENTRY_LOAD_CONFIG	10	Load Configuration directory	176
IMAGE_DIRECTORY_ENTRY_BOUND_IMPORT	11	Bound Import directory	184
IMAGE_DIRECTORY_ENTRY_IAT	12	Import Address Table	192
IMAGE_DIRECTORY_ENTRY_DELAY_IMPORT	13	Delay Import table	200
IMAGE_DIRECTORY_ENTRY_COM_DESCRIPTOR	14	COM descriptor table	208

FIGURE 4.6: Data Directory Entires[9]

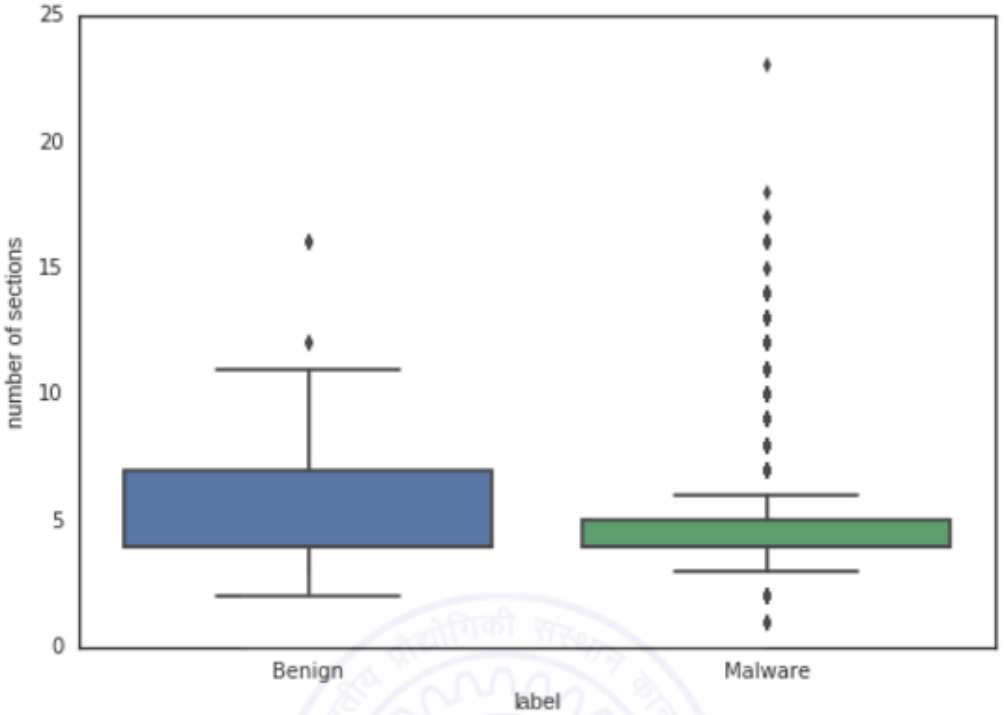

FIGURE 4.7: Distribution Of Number Of Sections

Sections are used to segment the entire file content into code, data and resources. According to the specification, the maximum number of allowed sections are 96. Most of the non-malicious executables uses small number of sections. In Figure 4.7, there are lot of outliers if we see the distribution for malware samples. They may use very low number of sections in case they are packed or can go upto 25 sections too. Number of sections has been useful in detection when combined with other potential indicators.

Figure 4.8 shows the range of sizes of the optional header in PE32 executables. Clearly there is a lot of difference in the sizes of the headers used by both the classes. Finally, we collected 50 features from PE Headers for detection. Table 5.1 shows some attributes extracted from the Headers with huge difference in their mean values for benign and malwares.

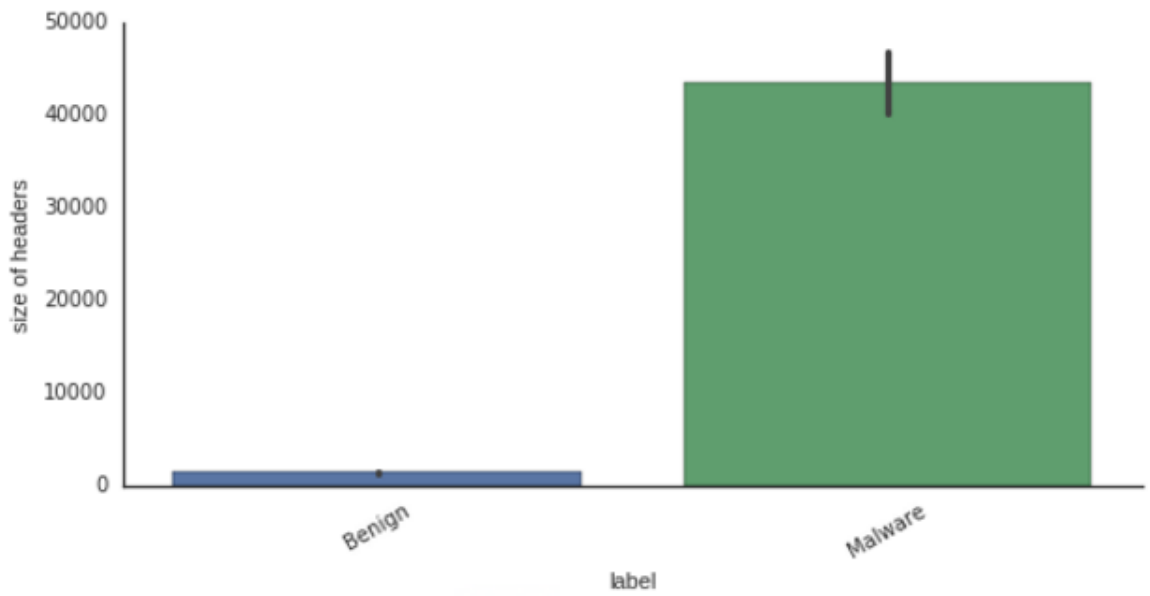

FIGURE 4.8: Distribution Of SizeOfHeaders Field

No.	Feature	Mean for Benign	Mean for Malware
1	Size of initialized data	1429198	4095
2	BaseOfCode	49392	252530
3	BaseOfData	592252	3113835
4	CheckSum	5252528	13164042
5	IMAGE_DIRECTORY_ENTRY_LOAD_CONFIG_size	1225	1013201
6	IMAGE_DIRECTORY_ENTRY_EXPORT_size	36	14383435
7	Loader flags	0	699220
8	Number of symbols	60	1413797
9	number_of_rva_and_sizes	16	985629
10	Size of headers	1519	43456
11	Size of uninitialized data	36	14383435
12	IMAGE_DIRECTORY_ENTRY_IAT_rva	759730	79655

TABLE 4.1: Mean value comparison for Various attributes in PE Headers

As shown in the table 5.1, the mean size of Initialized data in benign files is very high as compared to malwares. Malware files are found to use reasonably large number of symbols as compared to benign files.

4.1.1.2.2 Dealing with obfuscation : Entropy Feature Vectors

In this thesis, we have not manually unpacked any of the malware samples before doing any analysis on them. Most of the past work reported in the literature, either used all unpacked samples or have first unpacked them using publicly available packers. Malware authors have

been relying on compression or packing algorithms in order to hide executable's string and code data. These encryption algorithms convert all the original bytes of an executable into random looking bytes and thus making it very difficult for analysts to automate static analysis methods for malwares detection.

In order to deal with the packed executables, we have used entropy based features. Entropy is defined as the measure of of disorderness or uncertainty in the sequence of bytes. We have calculated entropy on the bytes of each of the sample and measured the disorder in the distribution of the bytes in the byte-level representation as a value between 0(Order) and 8(Random). We have applied sliding window algorithm in which we convert the bytes of malware into a series of entropy estimates $E = e_j : j = 1, ..., N$ where e_j is the entropy measured in each window of byte-code and N is the total number of windows. Finally entropy is calculated using the following Shanon's formula:

$$e_j = -\sum_{i=1}^{m} f(i) \log_2 f(i),$$

where $f(i)$ is the frequency of byte i in window j and m is the number of unique bytes in the window. We have used window size of 10000 Bytes and then computed standard statistical measures on the entropy estimates obtained using sliding window problem such as mean, variance, median and min-max. In addition, we have also added the entropy of whole byte-code as well as entropy of individual sections in the final feature set.

There is a feature in *pefile* Python module in which we ran all our malware samples against the signatures of publicly available packers which are frequently used by malware authors. Out of 11,347 malware samples used in our analysis, we found out that around 7000 samples were packed. So we had a fair mix of packed and unpacked binaries in our corpus.

4.1.1.2.3 String Based Features

Most of the authors have used printable string information extracted from executables for malware classification. The main problem while processing strings is that there is a lot of noisy data in the output along with some useful strings. Figure 4.9 shows the output of string command applied over a binary. It is clear that the binary is trying to search for a file using API's "FindFirstFileA" and "FindNextFileA" and then by using "CopyFileA", it will replace it with another file. Sometimes there IP addresses too in the output which could mean that it is a bot [6] and is trying to contact its controlling server using that IP.

```
CloseHandle
UnmapViewOfFile
IsBadReadPtr
MapViewOfFile
CreateFileMappingA
CreateFileA
FindClose
FindNextFileA
FindFirstFileA
CopyFileA
KERNEL32.dll
malloc
exit
MSVCRT.dll
_exit
_XcptFilter
__p__initenv
__getmainargs
_initterm
_setusermatherr
_adjust_fdiv
__p__commode
__p__fmode
__set_app_type
_except_handler3
_controlfp
_stricmp
kernel32.dll
kernel32.dll
.exe
C:\*
C:\windows\system32\kernel32.dll
Lab01-01.dll
C:\Windows\System32\Kernel32.dll
WARNING_THIS_WILL_DESTROY_YOUR_MACHINE
```

FIGURE 4.9: String command output on some executable

Since there is a lot of garbage in the output of strings utility, we have not used strings directly. In order to avoid over-fitting and reduce noise, only frequency bins related to the distribution of length of strings is used in our feature set.

4.1.1.3 Dynamic Feature Extraction

Features based on run-time behavior of executables are extracted from the analysis reports generated in the host machine after executing the sample inside a controlled environment. We generated all the reports by submitting the samples including both benign and malicious files to Cuckoo Sandbox [1]. Reports generated by the cuckoo sandbox are in JSON structure format as shown in Figure 4.10.

```
{
    "info": {
    },
    "signatures": [],
    "target": {
    },
    "virustotal": {
    },
    "network": {
    },
    "behavior": {
        "generic": [
            {
            },
            {
            }
        ],
        "apistats": {
        },
        "processes": [
        ],
        "processtree": [
        ],
        "summary": {
        }
    },
    "debug": {
    },
    "strings": [
    ],
    "metadata": {
        "output": {
            "pcap": {
                "basename": "dump.pcap",
                "sha256": "55d76708bf6e1005b0c9ac1244069df3739d9c3583f03650f581b68f3063467d",
                "dirname": ""
            }
        }
    }
}
```


FIGURE 4.10: Cuckoo Analysis Report in JSON Format

For each sample analyzed in the sandbox, the depicted structure will be used for further processing. Each analysis report contains the execution trace of a program which includes process tree, file and registry changes, memory dump and mutexes etc. Though there is a vast amount of information present in the report, we have limited ourselves to only three system resources : File, Registry and Mutexes. Our experiment shown that these are the most critical system resources used by malware authors for performing malicious activities and therefore, our dynamic features are based on the set of actions performed by malwares on these critical system resources.

4.1.1.3.1 File System Changes Related Features

Possible sets of action performed by a malware on file system can be :

- **Create** : Malware may create a new file/directory

- **Read** : Reading the content of any file on the system
- **Copy** : Copying some of the files to the directories created by the malware
- **Rename** : Changing the name of a file or a directory
- **Delete** : Deleting a file or an entire directory from the system

Each action is modeled as a feature by taking a count of how frequent an action is performed on the system. Along with this, we visualized the most frequent directories which are accessed by the malware and benign-ware as shown in Figure 4.11 and 4.12. We believed that how heavily a particular directory is being touched could also be used as a feature. So we included the count for the number of times a particular directory is touched (copy, created, renamed, deleted) as a feature and it is done for 7 number of directories namely 'c:\programfiles', "c:\programdata", "c:\recyclebin", "c:\windows\system32", "c:\python27", "c:\Users\ahuja\AppData\Local\Temp".

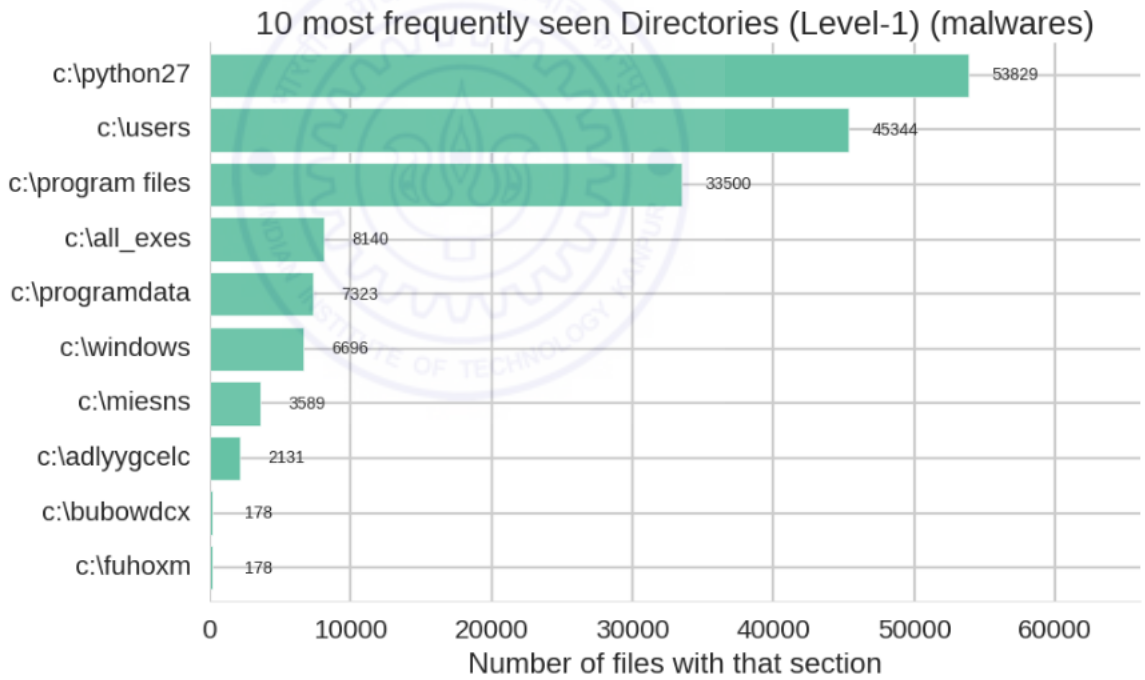

FIGURE 4.11: Frequent Directories Touched by malware

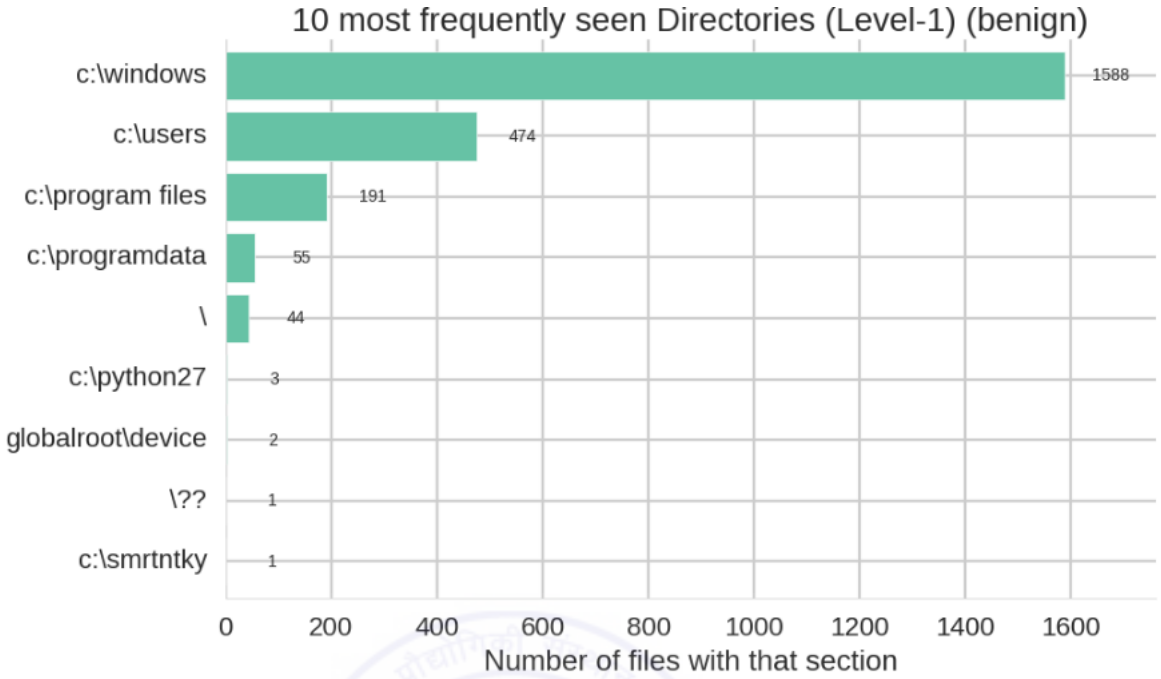

FIGURE 4.12: Frequent Directories Touched by benign-ware

We also observed that there are lots of different extensions created by malware and benign-ware as shown in Figure 4.13 and 4.14. After going through the visualizations, we decided to add the count for heavily created extensions namely "vir", "tmp", "dll", "txt", "py", "pyc", "exe", "mui", "cdf-ms", "ini", "xml", "lnk", "dat", "js", "json" and "db".

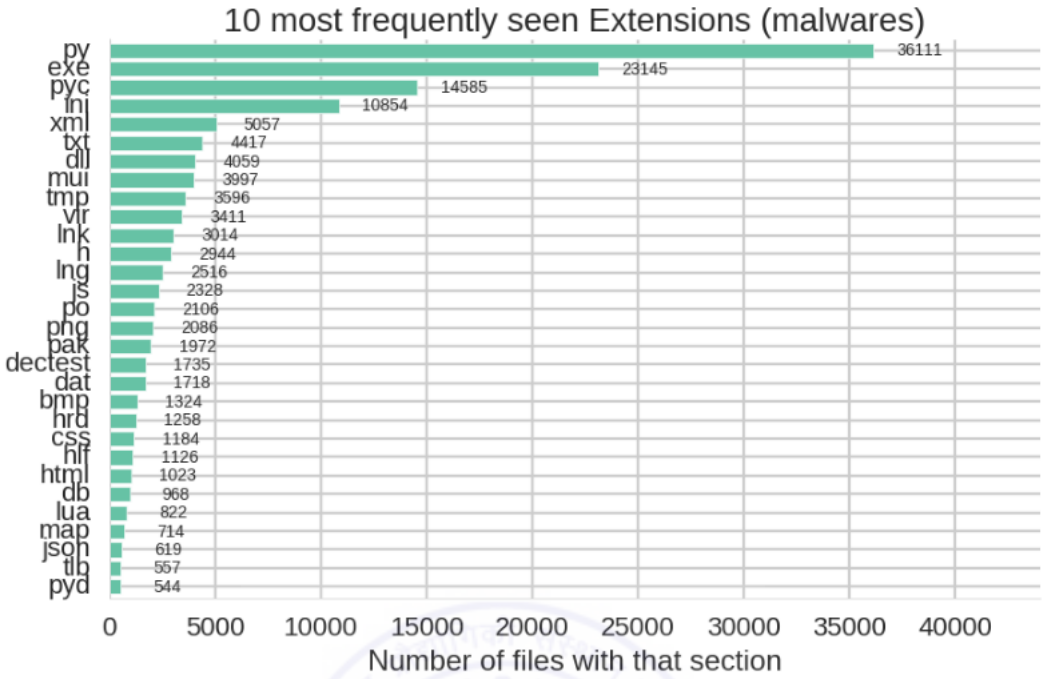

FIGURE 4.13: Extensions Touched by malware

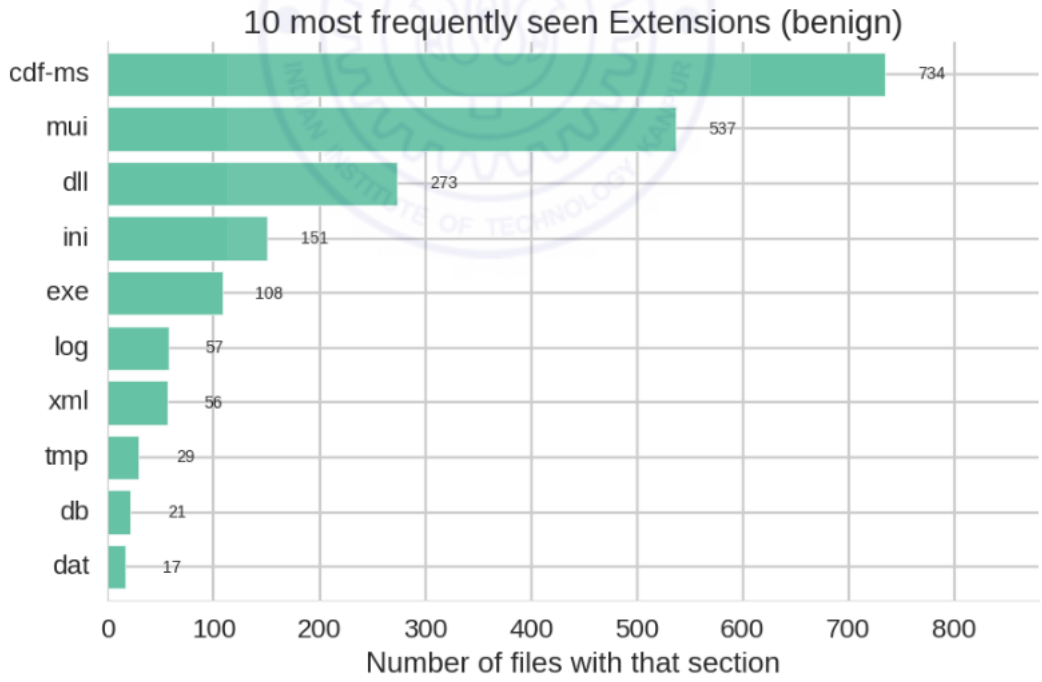

FIGURE 4.14: Extensions Touched by benign-ware

4.1.1.3.2 Windows Registry Based Features

In Windows, all the configuration information about user and system are stored in a database

called Registry Keys [11]. Windows continually references the information present in the registry as it contains profile of each user of the system as well as information about hardware and installed programs. There are malware which store their entire malicious payload in windows registry in order to ensure that they are executed at the startup. Various actions which can be performed on *registry* are :

- **Create** : Creating a new registry key or sub-key
- **Read** : Reading the content of any registry key or sub-key
- **Modify** : Modifying the value of registry key or sub-key
- **Delete** : Deleting an existing registry key or sub-key

As of now, we are only relying on how frequently these actions are performed by the malware and Benign-ware. We have included a count for each of these actions on the registry as a feature in our final feature set.

4.1.1.3.3 Mutex Based Features

According to MSDN documentation [8], *Mutex* is a synchronization object which is used to provide exclusive access to a shared resource in multi-threaded applications. If a thread has acquired a mutex, then the second thread who wants to acquire it has to be suspended until the first thread releases that mutex. Web-Browser is a very simple example of demonstrating how mutexes are used in the system. Every browser maintains a history file and when you have multiple windows open for browsing, each window which is basically a different process will try to update the history file in mutually exclusive manner by registering a mutex object with the history file.

Here is the list of some of the mutexes used by some legitimate software on Windows shown in Figure 4.15 below.

ContentIndex_Perf_Library_Lock_PID_b58
CtfmonInstMutexDefaultS-1-5-21-1960408961-725345543-1149019770-500
E7F93AFA-D81E-4635-8A95-8458F48D369D
ESENT_Perf_Library_Lock_PID_1bc
ISAPISearch_Perf_Library_Lock_PID_3d8
iTunes-{95FD314A-DC69-4e30-AFD8-3D2E80612344}
Lsa_Perf_Library_Lock_PID_8f0
McAfee.VS.OAS.1807E9718F2D4BB3B8CCDEA17EA6A8A4
McAfeeEngineReloadMutex
McAfeeVscanBofUpdateMutex
McSvHost_CAD0E02E86CD4436B6318C111B9092AC
mcupdmgr_CAD0E02E86CD4436B6318C111B9092AC
MSCTF.GCompartListMUTEX.DefaultS-1-5-21-1960408961-725345543-1149019770-500

FIGURE 4.15: Legitimate Mutexes

Here is a list of some of the mutexes used by malicious executables on Windows shown in Figure 4.16 below.

3f4bb7fc
6d5ac198
746bbf3569adEncrypt
__DDrawExclMode__
_SHuassist.mtx
AMResourceMutex2
ASPLOG
d242A3A092D94E907320A17430
DDrawDriverObjectListMutex
DDrawWindowListMutex
DENEK
DirectSound DllMain mutex (0x0000009C)
DirectSound DllMain mutex (0x000000C8)
DirectSound DllMain mutex (0x00000168)
DirectSound DllMain mutex (0x00000594)
DirectSound DllMain mutex (0x000005CC)
DirectSound DllMain mutex (0x000005DC)
DPSoundStreamMutex
HPWuSchedv.exeDm28sf0V@XK$NX8hOu
iemate_Crash
iemate_history

FIGURE 4.16: Non-legitimate Mutexes

After analyzing the mutexes accessed by both malware and benign-ware, some of the obvious differences include :

- Difference in the length of Mutex names
- Difference in the formatting of Mutex names
- Difference in the usage and distribution of special characters used in the Mutex names

In order to capture all the above mentioned differences, features related to mutexes include number of mutexes created and deleted per executable, average length of mutexes per executable, average and longest run of consecutive lowercase, uppercase, and digits from the mutexes name and finally entropy measures of the names as we did for dealing with the obfuscation by calculating entropies on the bytes of a file.

4.2 Machine Learning Classifiers: Scikit-Learn

We have evaluated the efficiency of integrated approach using popular Python based machine learning library Scikit-Learn. We have used various classification algorithms on our data set each of which is briefly described below :

- **K-Nearest Neighbors** : The main idea behind this algorithm is : if there are lot of samples in the feature space belonging to one class as compared to another, then this algorithm will label that area of feature space as belonging to that class. It will basically predict the class of new sample by finding the k-nearest neighbors in the training set. The distance which is generally used is the **Euclidean distance**, the sum of squares of the difference of each feature value between 2 samples :

  $$d(a,b) = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2 + \cdots + (x_n - y_n)^2}.$$

  The code snippet for using this algorithm in scikit-learn along with k-fold cross validation and parameter tuning is shown below :

```
from sklearn.neighbors import KNeighborsClassifier
clfknn = KNeighborsClassifier()
parameters = {"n_neighbors": range(2,40,2)}
```

```
bestknn, xtrain, yTrain, xtest, yTest = do_classify(clfknn, parameters,
    db_merge, merged_features, 'label', 'Benign', mask = mask, score_func =
    'f1')
```

- **Random Forests** : It is a meta-estimator which fits a number of decison trees classifiers and randomness is injected while building those trees. Following steps are repeated n times to improve prediction accuracy and control over-fitting :
  - Taking random sub-sample of all the data
  - Select m variables at random out of all M possible values independently for each node
  - Find the best split on the selected m variables. Build out the entire tree
  - Repeat

  For a new data point, we can run that point through all the trees constructed in the algorithm, get all the decisions and take a majority vote. We have optimized the *n_estimators* which is the number of trees in the forest. The code snippet for using this algorithm in scikit-learn along with k-fold cross validation and parameter tuning is shown below :

```
from sklearn.ensemble import RandomForestClassifier
clfForest = RandomForestClassifier()

parameters = {"n_estimators": range(1, 50)}
bestForest, xtrain, yTrain, xtest, yTest = do_classify(clfForest,
    parameters, db_merge, merged_features, 'label', 'Benign', mask = mask,
    score_func = 'f1')
```

- **XG-Boost** : It is one of the most popular technique used in kaggle competitions to achieve high classification accuracy. It is the parallel implementation of the gradient boosting tree classifier which most of the time produce much better performance than random forests. Boosting refers to the problem of producing very accurate prediction by combining the results from weak-learners. There are generally three elements which are used in this technique :
  - **Loss Function** : It generally depends on the type of problem to be solved. Regression might use square error while classification might use log-loss

- **Weak Learner** : A weak learner is defined as the one whose performance is slightly better than the random choice. Decision trees are the week learners and are constructed greedily choosing the best split points to minimize the loss. In each iteration, next decision tree is built by giving higher weight to misclassified points by the previous tree.
- **Additive Model** : Trees are added in each iteration and existing trees are not changed. Generally, gradient descent is used to minimize a set of parameters such as coefficients in regression or weights in neural networks. Here, we are parameterizing a tree and we must add a tree to the model that reduces the loss. Fixed number of trees are added in the model or training stops once loss reaches an acceptable level.

We have optimized this algorithm for various parameters including gamma, max_depth, n_estimators, min_child_weight and subsample. The code snippet for using this algorithm in scikit-learn along with k-fold cross validation and parameter tuning is shown below :

```
import xgboost as xgb

gbm = xgb.XGBClassifier()
parameters = {"gamma":[0.1, 0.3, 0.5, 0.7, 1], "max_depth":[3, 5, 7, 9, 12,
    15, 17, 25], "n_estimators":[10,50,100,200,300],
              "min_child_weight":[1, 3, 5, 7], "subsample":
    [0.5,0.6,0.7,0.8,0.9]}
bestgbm, xtrain, yTrain, xtest, yTest = do_classify(gbm, parameters,
    db_merge, merged_features, 'label', 'Benign', mask = mask, score_func =
    'f1')
```

4.3 Summary

This section presented the main architecture and all the analysis done in our malware detection system. Here, the features from both static as well as dynamic analysis to be used, were chosen using some data visualizations. PE headers, entropy measures on bytes of a file and frequency bins related to length of strings were chosen in static analysis. Directory usage, extensions created, mutexes and registry accesses were chosen for dynamic analysis. After feature extraction part, k-nearest neighbors, Random forest and Gradient Boosting techniques were used for data modeling. Hyper-parameters were optimized using Grid search which exhaustively generates

candidates from a grid of parameter values and k-fold cross validation is also performed in order to avoid over-fitting.

Chapter 5

Experimental Results

5.1 Malware and Benignware Dataset

For malware, we approached VirusTotal, which is one of the biggest platforms where we can analyze any malware sample, and they allowed us to use their paid VirusTotal Intelligence Service for 2 months. We collected around 15,000 malware samples from their platform. Final working data set include 11347 malware as some of the samples either could not be parsed by *pefile* Python module or some of them could not be run inside Virtual Machine due to some errors.

For benign executables, we used a simple python script to scrape all the valid executables from a freshly installed windows 7 and windows XP on a virtual machine. We were able to collect around 773 files. 1000 Benign files were downloaded from the VirusTotal intelligence search by restricting the number of positives for all the anti-virus vendors to be zero.

5.2 Evaluation Metric

Various metrics which are used to evaluate the performances of various classifiers are given below :

5.2.1 Confusion Matrix

It is a table which is used to describe the performance of a classifier on the holdout test set which is locked away until the model calibration process is finished to prevent underestimation of the true model error. There are 4 metrics involved in a confusion matrix :

- **True Positives (TP)** : It is the case when a sample is correctly predicted as benign.
- **True Negatives (TN)** : It is the case when a sample is correctly predicted as malware.
- **False Positives (FP)** : It is the case when a sample is incorrectly predicted as benign.
- **False Negatives (FN)** : It is the case when a sample is incorrectly predicted as malware.

Skeleton confusion matrix for a binary classifier is shown below :

		Predicted	
		Positive	**Negative**
Observed	**Positive**	TP (# of TPs)	FN (# of FNs)
	Negative	FP (# of FPs)	TN (# of TNs)

FIGURE 5.1: Binary Confusion Matrix

5.2.2 Additional Metrics

There are few other metrics which we have used for evaluation. These are :

- **True Positive Rate (TPR)** : It is defined as the ratio of the number of correct positive predictions to the total number of positives in the test set. Low value of TPR indicated large number of False Negatives.

$$TPR = TP/(TP + FN)$$

- **False Positive Rate (FPR)** : It is defined as the ratio of the number of negative samples being categorized as positive to the total number of negative samples in the test set.

$$FPR = FP/(TN + FP)$$

- **Precision** : It is defined as the ratio of the the number of correct positive predictions to the total number of positive predictions made by classifier. It is also called as Positive Predicted Value (PPV). A low precision indicates large number of false positives.

$$PPV = TP/(TP + FP)$$

- **F Measure** : When there are imbalanced classes in the dataset, classifier would achieve high accuracy by guessing the most common class. F-score tends to favor classifiers that are strong in both precision as well as recall rather than the classifiers which emphasize on one at the cost of other. It is defined as the harmonic mean of precision and recall.

$$F - M = (2 * TPR * PPV)/(TPR + PPV)$$

5.3 Training and Testing

Our experimental environment which carried out all the training and testing was set up on a Ubuntu 16.04 LTS machine with 16 GB RAM and Intel i7 octa-core processor. Our training set consists of 70% of the data while remaining 30% goes into the testing set. For each of the models, we have used 10-fold cross validation as depicted in Figure 5.2 in order to minimize the risk of over-fitting and to ensure that model generalizes well. We have used GridSearchCV functionality from scikit-learn for choosing the best hyper-parameters of model.

5.3.1 The entire description of K-fold Cross-validation

- Create n_folds partitions of the training data
- We then train on n_folds -1 of these partitions, and test on the remaining partition. There are n_folds such combinations of partitions (or folds), and thus we obtain n_fold risks
- We average the error or risk of all such combinations to obtain, for each value of parameter supplied d, R_{dCV}
- We move on to the next value of d, and repeat 3
- Find the optimal value of d that minimizes risk $d = *$.
- We finally use that value to make the final fit in $\mathscr{H}_*$ on the entire old training set.

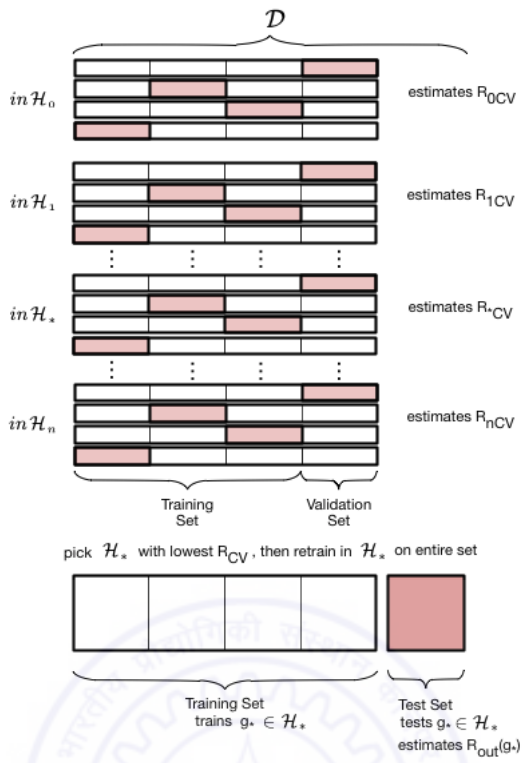

FIGURE 5.2: K-Fold Cross Validation

Here is the code snippet for performing the 10-fold cross validation along with Parameter tuning of models :

```
def cv_optimize(clf, parameters, X, y, n_jobs=1, n_folds=5, score_func=None):
    if score_func:
        gs = GridSearchCV(clf, param_grid=parameters, cv=n_folds, n_jobs=n_jobs,
    scoring=score_func)
    else:
        gs = GridSearchCV(clf, param_grid=parameters, n_jobs=n_jobs, cv=n_folds)
    gs.fit(X, y)
    print "BEST", gs.best_params_, gs.best_score_, gs.grid_scores_
    best = gs.best_estimator_
    return best
```

The obtained results show that the hybrid method, where both static and dynamic feature are used in integration, achieves best results with Gradient Boosting approach implemented in XGboost as shown in the table 5.2. Table 5.1 shows the results when only static features comprised of PE headers, entropy measures and string histograms were used for classification. Xg-Boost is the best performer among the classifiers. Results from table 5.1 and 5.2 indicate that xg-Boost

which is based on Gradient Boosting is best equipped to detect malwares in our dataset. However, with each of classifiers KNN, RF and XG, detection accuracy got improved by combining the features from the analysis.

Class	XG-Boost				Random Forest				KNN			
	TPR	FPR	Pr	FM	TPR	FPR	Pr	FM	TPR	FPR	Pr	FM
Benign	0.908	0.01	0.905	0.906	0.893	0.001	0.982	0.935	0.877	0.03	0.801	0.837
Malware	0.984	0.09	0.985	0.984	0.982	0.10	0.888	0.932	0.969	0.122	0.982	0.975
Average	0.973	0.03	0.984	0.972	0.970	0.086	0.900	0.932	0.956	0.109	0.957	0.956

TABLE 5.1: Test Results with static features set (PE Headers + Entropy + String Histograms)

Class	XG-Boost				Random Forest				KNN			
	TPR	FPR	Pr	FM	TPR	FPR	Pr	FM	TPR	FPR	Pr	FM
Benign	0.960	0.01	0.920	0.948	0.954	0.01	0.903	0.927	0.916	0.02	0.814	0.861
Malware	0.987	0.039	0.994	0.991	0.985	0.046	0.993	0.988	0.971	0.08	0.988	0.979
Average	0.986	0.03	0.986	0.986	0.980	0.04	0.980	0.979	0.963	0.071	0.964	0.963

TABLE 5.2: Test Results with integrated feature set

Looking at the final weighted average accuracy of the classifier, it performs satisfactory and is comparable to the past results in the hybrid approach where less amount of samples were used. We have also got good F-Measure of 0.986 which hides the fact that TPR for benign samples is not that good. The main goal of this thesis was to achieve High TPR for the malwares while not predicting too many benign as malwares. The confusion matrix for Best detection which is obtained by XG-Boost using integrated featured feature set is depicted in Figure 5.3.

```
Confusion Matrix Stats
malicious/malicious: 99.41% (3388/3408)
malicious/benign: 0.59% (20/3408)
benign/malicious: 7.95% (42/528)
benign/benign: 92.05% (486/528)
```

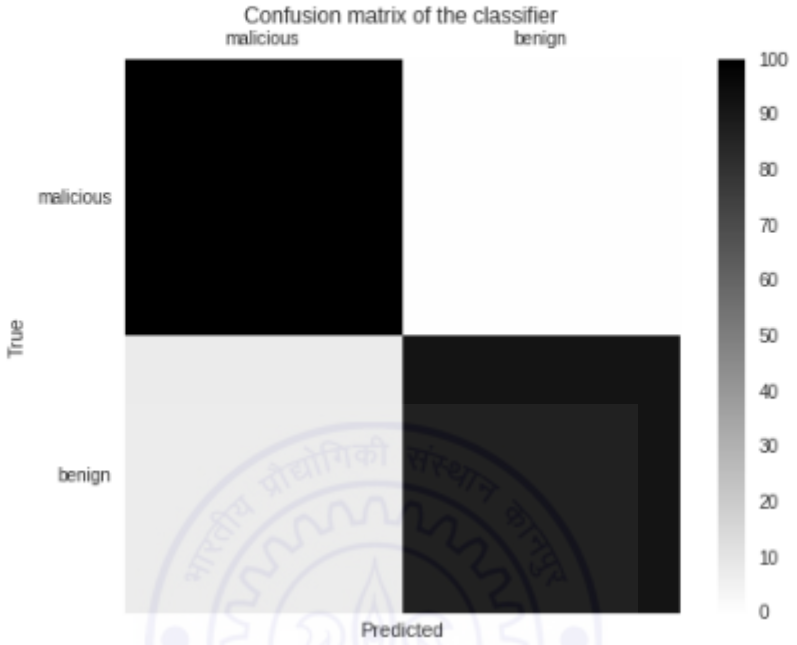

FIGURE 5.3: Confusion Matrix For XG-Boost on integrated Feature Set

It can be observed that for the confusion matrix, the amount of false negatives for malware is quite low which also explains high TPR for malware. Only 20 sample out of 3408 malware go undetected. However, the amount of false negatives for benign files is bit too high compared to the total number of benign samples used which also explains low TPR.

5.4 Comparison To Existing Approaches

In this section, we will compare our work with the authors who have used hybrid analysis for malware classification. Table 5.3 shows the comparison results. Rafiqul et al. (2013) achieved a good accuracy but their approach used about 300 times the size of features space we chose and used all the unpacked binaries. A. Salim et al. (2014) achieved a good accuracy but samples used were very less and feature space was also very high dimensional. They also manually unpacked the binaries before doing feature extraction. Al-Nemrat et al. (2016) reduced the feature space considerably but the number of samples used was very less and all the samples were unpacked binaries. We achieved a comparable weighted accuracy of 98.62 % by using only

Authors	# of Features	Accuracy	# of Samples Used	Features Used
Rafiqul et al. (2013)	498 72,259	97.05%	541 Benign 2398 malwares	**Static** : PSI + FLF **Dynamic** : API calls + Parameters
A. Salim et al. (2014)	7253 9094	98.71 %	490 Benign 997 malwares	**Static** : PSI **Dynamic** : 3,4 API Calls Gram
Al-Nemrat et al. (2016)	289 636	99.20%	1515 malwares	**Static** : PSI + FLF **Dynamic** : System Changes
Ours	188 39	98.62 %	1773 Benign 11,347 malwares	**Static** : PE Headers + STR + ENT **Dynamic** : System Changes

TABLE 5.3: Past work done in Hybrid Analysis

188 static and 39 dynamic features and the amount of samples used were also quite high as compared to the above approaches. We also did not unpack any of the binaries before starting our analysis. Entropy measures were used to handle obfuscation in our analysis.

Conclusion : We were able to achieve our objectives of scalability by reducing the feature space as well as more robust detection by using more amount of samples in our analysis

Chapter 6

Scope and Future Work

6.1 Multi-Path Execution in Cuckoo sandbox

As of now, we are relying on the single execution path of executables inside Cuckoo sandbox. The problem is that behavior reports generated from single run of executable in sandbox could miss many of the malicious functionalities of malware. In order to address the problem of test coverage, a separate module can be integrated with Cuckoo sandbox so that more comprehensive reports can be generated. Work done by [21] on this problem was integrated into ANUBIS, the malware analysis platform that were hosted at UC Santa Barbara. We can not get to use ANUBIS in our thesis as this project retired couple of years ago and has been commercialized into a company, Lastline.

6.2 Performing Classification of malware families

The problem of malware detection which we addressed in this thesis can be further extended to classifying malware into their respective families. A malware family refers to a group of malware that originates from same source code and are divided into updated versions with slightly different behaviors. Once we have more amount of samples from multiple families, we can use different feature representations and deep learning models for effective malware classification along with detection.

6.3 Providing Malware detection and classification as a cloud based service

Considering the amount of computing resources required for static and dynamic analysis, our proposed malware detection system can be deployed in the Cloud. Ordinary users can analyze any executable for our malware detection system through cloud service.

Appendix A

Appendix A

Code snippet for Strings utility and histograms generation in Python is shown below:

```
all_binary_path = ["/home/ahuja/Thesis_Work/Dynamic_binaries/Benign",
    "/home/ahuja/Thesis_Work/Clustering","/home/ahuja/Thesis_Work/Clustering_remaining",
    "/home/ahuja/Thesis_Work/Malicious"]

string_features_list = []
for paths in all_binary_path:
    for root, dirs, files in os.walk(paths):
        for f in files:
            string_dict = get_strings_histograms(os.path.join(root, f))
            if string_dict is not None:
                string_features_list.append(string_dict)

with open('Strings_', 'w') as fout:
    json.dump(string_features_list, fout)
fout.close()
```

Code snippet for using pefile in Python is shown below:

```
def open_using_pefile(input_name):
        ''' Open the PE File using the Python pefile module. '''
        try:
            pe = pefile.PE(input_name)
        except (PEFormatError,AttributeError) as e:
            return None
        return pe
```

```
# Load up all our files
def load_files(file_list,Y):
    features_list = []
    for filename in file_list:
        pe_handle = extract_Feature_2(open_using_pefile(filename), Y, filename)
        if pe_handle is not None:
            features_list.append(pe_handle)
    return features_list
```

Following code snippet does the job of adding all the malwares to the database of Cuckoo.

```
import subprocess, shlex, os, time

bash_prefix = "python /home/ahuja/cuckoo/utils/submit.py"
Malicious_remain = "/home/ahuja/Thesis_Work/Malicious/"

for root, dirs, files in os.walk(Malicious_remain):
    for f in files:
        filename = os.path.join(root, f)
        bash_line = bash_prefix + " " + filename
        args = shlex.split(bash_line)
        p = subprocess.Popen(args)
        time.sleep(1)
```

Here is the code snippet for checking whether a sample is packed or not :

```
count_packed = 0
import peutils
signatures = peutils.SignatureDatabase('/home/ahuja/Thesis_Work/userdb.txt')

for paths in Malicious_files_Path:
    for root, dirs, files in os.walk(paths):
        for f in files:
            pe_handle = open_using_pefile(os.path.join(root, f))
            if pe_handle is not None:
                matches = signatures.match(pe_handle, ep_only = True)
                if matches is not None:
                    count_packed += 1
```

```
print "Packed Executables", count_packed
```

Here is the code snippet which we have used to calculate entropy estimates on byte-code of malwares using sliding window with a step size of 100 Bytes :

```
def sliding_window(data, block_size, step, counts, ent):

    entropy = 0.0
    for i in xrange(block_size):
        counts[data[i]] += 1

    for i in xrange(counts.shape[0]):
        if counts[i] > 0:
            entropy += - (counts[i] / block_size) * np.log2(counts[i] /
    block_size)

    ent[0] = entropy

    for i in xrange(1, data.shape[0] - block_size):

        dec = counts[data[i - 1]]
        inc = counts[data[i + block_size - 1]]

        counts[data[i - 1]] -= 1
        counts[data[i + block_size - 1]] += 1

        entropy -= -dec / block_size * np.log2(dec / block_size)
        if dec > 1:
            entropy += -(dec - 1) / block_size * np.log2((dec - 1) / block_size)

        if inc > 0:
            entropy -= -inc / block_size * np.log2(inc / block_size)

        entropy += - (inc + 1) / block_size * np.log2((inc + 1) / block_size)

        if i % step == 0:
            ent[i / step] = (entropy)
```

Here is the code snippet for scraping all the valid executables from freshly installed Windows

```
#!/usr/bin/python
import os, shutil

# The top argument for walk
topdirs = ['C:\\Windows', 'C:\\Program\ Files', 'C:\\Users']
copy_dir = 'C:\\All_exes'
# The extension to search for
exten = '.exe'

for topdir in topdirs:
        for dirpath, dirnames, files in os.walk(topdir):
                for name in files:
                        if name.lower().endswith(exten):
                                src_file = '%s' % os.path.join(dirpath, name)
                                shutil.copy(src_file, copy_dir)
```

Here is the code snippet for static feature extraction

```
def extract_Feature_2(pe,Y, filename):
    if pe is None:
        return None

    extracted_features = {}
    try:

        extracted_features['sha256'] = hashlib.sha256(pe.__data__).hexdigest()
        extracted_features['Y'] = Y
        # File_Header Features
        extracted_features['machine'] = pe.FILE_HEADER.Machine
        extracted_features['number of sections'] =
    pe.FILE_HEADER.NumberOfSections
        extracted_features['compile date'] = pe.FILE_HEADER.TimeDateStamp
        extracted_features['pointer to symbol table'] =
    pe.FILE_HEADER.PointerToSymbolTable
        extracted_features['number of symbols'] = pe.FILE_HEADER.NumberOfSymbols
```

```
    extracted_features['size of optional header'] =
pe.FILE_HEADER.SizeOfOptionalHeader
    extracted_features['characteristics'] = pe.FILE_HEADER.Characteristics


    # Optional Header Features
    extracted_features['magic'] = pe.OPTIONAL_HEADER.Magic
    extracted_features['major linker version'] =
pe.OPTIONAL_HEADER.MajorLinkerVersion
    extracted_features['minor linker version'] =
pe.OPTIONAL_HEADER.MinorLinkerVersion
    extracted_features['size init data'] =
pe.OPTIONAL_HEADER.SizeOfInitializedData
    extracted_features['size uninit data'] =
pe.OPTIONAL_HEADER.SizeOfUninitializedData
    extracted_features['section alignment'] =
pe.OPTIONAL_HEADER.SectionAlignment
    extracted_features['file alignment'] = pe.OPTIONAL_HEADER.FileAlignment
    extracted_features['major operating system version'] =
pe.OPTIONAL_HEADER.MajorOperatingSystemVersion
    extracted_features['minor operating system version'] =
pe.OPTIONAL_HEADER.MinorOperatingSystemVersion
    extracted_features['major image version'] =
pe.OPTIONAL_HEADER.MajorImageVersion
    extracted_features['minor image version'] =
pe.OPTIONAL_HEADER.MinorImageVersion
    extracted_features['major subsystem version'] =
pe.OPTIONAL_HEADER.MajorSubsystemVersion
    extracted_features['minor subsystem version'] =
pe.OPTIONAL_HEADER.MinorSubsystemVersion
    extracted_features['size of headers'] = pe.OPTIONAL_HEADER.SizeOfHeaders
    extracted_features['subsystem'] = pe.OPTIONAL_HEADER.Subsystem
    extracted_features['dll charactersitics'] =
pe.OPTIONAL_HEADER.DllCharacteristics
    extracted_features['loader flags'] = pe.OPTIONAL_HEADER.LoaderFlags
    extracted_features['number_of_imports'] = len(pe.DIRECTORY_ENTRY_IMPORT)
    extracted_features['AddressOfEntryPoint'] =
pe.OPTIONAL_HEADER.AddressOfEntryPoint
    extracted_features['SizeOfHeaders'] = pe.OPTIONAL_HEADER.SizeOfHeaders
    extracted_features['CheckSum'] = pe.OPTIONAL_HEADER.CheckSum
```

```
        extracted_features['size of stack reserve'] =
    float(pe.OPTIONAL_HEADER.SizeOfStackReserve)
        extracted_features['size of stack commit'] =
    float(pe.OPTIONAL_HEADER.SizeOfStackCommit)
        extracted_features['size of heap reserve'] =
    float(pe.OPTIONAL_HEADER.SizeOfHeapReserve)
        extracted_features['size of heap commit'] =
    float(pe.OPTIONAL_HEADER.SizeOfHeapCommit)
        extracted_features['image_base'] = float(pe.OPTIONAL_HEADER.ImageBase)
        extracted_features['Size_image'] = pe.OPTIONAL_HEADER.SizeOfImage
        extracted_features['BaseOfCode'] = pe.OPTIONAL_HEADER.BaseOfCode
        extracted_features['BaseOfData'] = pe.OPTIONAL_HEADER.BaseOfData
        extracted_features['number_of_rva_and_sizes'] =
    pe.OPTIONAL_HEADER.NumberOfRvaAndSizes
        extracted_features['number_of_IAT_entires'] =
    (pe.OPTIONAL_HEADER.DATA_DIRECTORY[12].Size)/4

        for index in range(15):
            extracted_features[pe.OPTIONAL_HEADER.DATA_DIRECTORY[index].name +
    '_rva']  = pe.OPTIONAL_HEADER.DATA_DIRECTORY[index].VirtualAddress
            extracted_features[pe.OPTIONAL_HEADER.DATA_DIRECTORY[index].name +
    '_size'] = pe.OPTIONAL_HEADER.DATA_DIRECTORY[index].Size
        extracted_features.update(get_entropy_features(filename))

    except AttributeError:
         return None

    return extracted_features


def prepare_pandas(feature_list):
    final_features = []

    for f in feature_list:
        feat_categ = defaultdict(lambda: 0)
        feat_categ["Y"] = f["Y"]
        feat_categ["sha256"] = f["sha256"]
        count_lower = count_upper = count_digit = 0
        lower_max = upper_max = digit_max = lower_mean = upper_mean = digit_mean
    = 0
        max_entropy = 0
```

```
    for mutexes in f["mutex"]:
        cn_lowercase_runs, cn_uppercase_runs, cn_digit_runs =
extract_character_info(mutexes)

        if entropy(mutexes) > max_entropy:
            max_entropy = entropy(mutexes)

        if len(cn_uppercase_runs) > 0 and np.max(cn_uppercase_runs) >
upper_max:
            upper_max = np.max(cn_uppercase_runs)

        if len(cn_lowercase_runs) > 0 and np.max(cn_lowercase_runs) >
lower_max:
            lower_max = np.max(cn_lowercase_runs)

        if len(cn_digit_runs) > 0 and np.max(cn_digit_runs) > digit_max:
            digit_max = np.max(cn_digit_runs)

        if len(cn_uppercase_runs) > 0:
            upper_mean += np.sum(cn_uppercase_runs)
            count_upper += len(cn_uppercase_runs)

        if len(cn_lowercase_runs) > 0:
            lower_mean += np.sum(cn_lowercase_runs)
            count_lower += len(cn_lowercase_runs)

        if len(cn_digit_runs) > 0:
            digit_mean += np.sum(cn_digit_runs)
            count_digit += len(cn_digit_runs)

    feat_categ["lower_max"] = lower_max
    feat_categ["upper_max"] = upper_max
    feat_categ["digit_max"] = digit_max
    feat_categ["entropy_mutex"] = max_entropy

    if count_lower:
        feat_categ["lower_mean"] = lower_mean/float(count_lower)
    else:
        feat_categ["lower_mean"] = 0
    if count_upper:
        feat_categ["upper_mean"] = upper_mean/float(count_upper)
```

```
        else:
            feat_categ["upper_mean"] = 0
        if count_digit:
            feat_categ["digit_mean"] = digit_mean/float(count_digit)
        else:
            feat_categ["digit_mean"] = 0


        feat_categ["count_mutex:"] = len(f["mutex"])
        feat_categ["files_operations:"] = f["files_operations"]
        f["file_copied"] = flatten_list(f["file_copied"])
        for oper in categ_oper:
            for files in f[oper]:
                #feat_categ["categ:" + __normalise_string(files)] = 1
                if "file" in oper:
                    ext = extension_count(__normalise_string(files))
                    if ext in extensions_touched:

    feat_categ[extension_count(__normalise_string(files))] += 1

                    for prefixes in directory_features:
                        if __normalise_string(files).startswith(prefixes):

    feat_categ[directory_features[directory_features.index(prefixes)]] += 1
                            break

            feat_categ["count_" + oper] = len(f[oper])

        for oper in dynamic_oper_counts:
            feat_categ["count_" + oper] = len(f[oper])

        final_features.append(copy.copy(feat_categ))
    return final_features


import subprocess
str_features = ['string_len_counts_' + str(x) for x in range(2,100)] +
    ['sha_256','string_len_counts_1_10','string_len_counts_10_30',
    'string_len_counts_30_60', 'string_len_counts_60_90',
    'string_len_counts_100']
```

```
def get_strings_histograms(filename):
    str_output = subprocess.check_output(['strings', filename]).split('\n')
    default_dict_counter = dict.fromkeys(str_features, 0)
    for s in str_output:
        if len(s):
            if len(s) < 100:
                default_dict_counter['string_len_counts_' + str(len(s))] += 1
            else:
                default_dict_counter['string_len_counts_100'] += 1

    default_dict_counter['string_len_counts_1_10'] =
    sum([default_dict_counter['string_len_counts_' + str(c)] for c in
    range(1,10)])
    default_dict_counter['string_len_counts_10_30'] =
    sum([default_dict_counter['string_len_counts_' + str(c)] for c in
    range(10,30)])
    default_dict_counter['string_len_counts_30_60'] =
    sum([default_dict_counter['string_len_counts_' + str(c)] for c in
    range(30,60)])
    default_dict_counter['string_len_counts_60_90'] =
    sum([default_dict_counter['string_len_counts_' + str(c)] for c in
    range(60,90)])
    default_dict_counter['sha_256'] = hashlib.sha256(open(filename,
    'rb').read()).hexdigest()

    return default_dict_counter
```

Here is the code snippet for Dynamic feature extraction

```
def extract_dynamic_features(json_report, Sample):
    features = {}
    features["mutex"] = report.get("behavior", {}).get("summary",
    {}).get("mutex", [])
    if Sample is "Benign":
        features["Y"] = 1
    else:
        features["Y"] = 0

    features["sha256"] = report.get("target", {}).get("file", {}).get("sha256",
    [])
```

```
for oper in categ_oper:
    features[oper] = report.get("behavior", {}).get("summary", {}).get(oper,
[])

for oper in dynamic_oper_counts:
    features[oper] = report.get("behavior", {}).get("summary", {}).get(oper,
[])

file_operations = report.get("behavior", {}).get("summary", {})\
                    .get("file_read", []) + \
                    report.get("behavior", {}).get("summary", {})\
                    .get("file_written", []) + \
                    report.get("behavior", {}).get("summary", {})\
                    .get("file_deleted", []) + \
                    flatten_list(report.get("behavior", {}).get("summary",
{})\
                    .get("file_copied", [])) + \
                    flatten_list(report.get("behavior", {}).get("summary",
{})\
                    .get("file_moved", [])) + \
                    report.get("behavior", {}).get("summary", {})\
                    .get("file_recreated", []) + \
                    report.get("behavior", {}).get("summary", {})\
                    .get("file_opened", []) + \
                    report.get("behavior", {}).get("summary", {})\
                    .get("file_exists", []) + \
                    report.get("behavior", {}).get("summary", {})\
                    .get("file_failed", [])


features["files_operations"] = len(set(file_operations))


api_sequences = []
for proc in report.get("behavior", {}).get("processes", []):
    for call in proc["calls"]:
         api_sequences.append(call["api"])

api_stats = report.get("behavior", {}).get("apistats", {})
features["api_stats"] = {}
```

```
    for api in api_stats:
        for x in api_stats[api]:
            if x in features["api_stats"]:
                features["api_stats"][x] += api_stats[api][x]
            else:
                features["api_stats"][x] = api_stats[api][x]

    return features
```

Here is the code snippet for stacking technique to combine various classifiers

```
from sklearn.cross_validation import StratifiedKFold
from sklearn.metrics import accuracy_score
import itertools


subdf=db_merge[merged_features]
X=subdf.values
y=(db_merge['label'].values=='Benign')*1
X_train, X_test, y_train, y_test = X[mask], X[~mask], y[mask], y[~mask]

skf = list(StratifiedKFold(y_train, 5))

print "Creating train and test sets for blending."

clfs = [bestknn, bestclftree1, bestAda, bestGB, bestGNB, calibrated_random,
    calibrated_xg]
blend_size = 3
accuracy_scores = []
clf_list = list(itertools.combinations(list(xrange(len(clfs))), blend_size))
for clsfr in clf_list:
            dataset_blend_train = np.zeros((X_train.shape[0], blend_size))
            dataset_blend_test = np.zeros((X_test.shape[0], blend_size))
            for j, clf_num in enumerate(list(clsfr)):
                dataset_blend_test_j = np.zeros((X_test.shape[0], len(skf)))
                print clfs[clf_num]
                for i, (train, test) in enumerate(skf):
                    X_Train = X_train[train]
                    y_Train = y_train[train]
                    X_Test = X_train[test]
                    y_Test = y_train[test]
```

```
                clfs[clf_num].fit(X_Train, y_Train)
                y_submission = clfs[clf_num].predict_proba(X_Test)[:, 1]
                dataset_blend_train[test, j] = y_submission
                dataset_blend_test_j[:, i] =
clfs[clf_num].predict_proba(X_test)[:, 1]
            dataset_blend_test[:, j] = dataset_blend_test_j.mean(1)
        print "Blending."
        parameters = {"gamma":[0.1, 0.3, 0.5, 0.7, 1], "max_depth":[3, 5, 7,
9, 12, 15, 17, 25], "n_estimators":[10,50,100,200,300],
          "min_child_weight":[1, 3, 5, 7], "subsample":
[0.5,0.6,0.7,0.8,0.9]}
        bestgbm =
cv_optimize(xgb.XGBClassifier(),parameters,dataset_blend_train, y_train)
        y_submission_labels = bestgbm.predict(dataset_blend_test)
        print "Blending Accuracy %f" % accuracy_score(y_test,
y_submission_labels)
        print confusion_matrix(y_test, y_submission_labels)
```

Bibliography

[1] (2016). Cuckoo sandbox. https://www.cuckoosandbox.org/.

[2] (2017). Av-test security institute. https://www.av-test.org/en/statistics/malware/.

[3] (2017). Cs109 harward. https://github.com/cs109/2015lab5/blob/master/LearningModels.ipynb.

[4] (2017). Cuckoo architecture. http://docs.cuckoosandbox.org/en/latest/introduction/what/.

[5] (2017). Interactive disassembler (ida). https://en.wikipedia.org/wiki/Interactive_Disassembler.

[6] (2017). Internet bot. https://en.wikipedia.org/wiki/Internet_bot.

[7] (2017). Microsoft detours. https://www.microsoft.com/en-us/research/project/detours/#.

[8] (2017). Mutex objects. https://msdn.microsoft.com/en-us/library/windows/desktop/ms684266(v=vs.85).aspx.

[9] (2017). Pe header. https://en.wikibooks.org/wiki/X86_Disassembly/Windows_Executable_Files.

[10] (2017). Virus total. https://virustotal.com/en/documentation/.

[11] (2017). Windows registry. https://www.microsoft.com/resources/documentation/windows/xp/all/proddocs/en-us/regedit_o.mspx?mfr=true.

[12] Cheng Wang, J. P. and Zhao, R. (2009). Using api sequence and bayes algorithm to detect suspicious behavior. In *In Proc. of the Int. Conf. on Communication Software and Networks*, page 544–548. IEEE.

[13] E. Gandotra, D. B. and Sofat., S. (2014). Integrated framework for classification of malwares. In *In Proc. of the 7th Int. Conf. on Security of Information Networks*, page 417. ACM.

[14] I. Santos, J. Devesa, F. B. J. N. and Bringas., P. G. (2013). Opem: A static-dynamic approach for machine learning based malware detection. In *In In Proc. of the Int. Joint Conf. CISIS'12-ICEUTE'12-SOCO'12*, pages 271–280.

[15] I. Santos, F. Brezo, J. N. Y. K. P. B. S. C. L. and Bringas., P. G. (2010). Opcode-sequence-based malware detection. In *In Proc. of the 2nd int. Conf. on Engineering Secure Software and Systems*, page 35–43. IEEE.

[16] Kolter, J. and Maloof, M. (2004). Learning to detect malicious executables in the wild. In *In Proc. of the 10th ACM Int. Conf. on Knowledge Discovery and Data Mining*, page 470–478. IEEE.

[17] Lobo D, Watters P, W. X. (2010). Rbacs rootkit behavioral analysis and classification system. In *In: Proceedings of the third international conference on knowledge discovery and data mining, 2010*, page 75–80.

[18] M. Chandramohan, H. B. K. T. and Shar., L. K. (2012). Scalable malware clustering through coarse-grained behavior modeling. In *In Proc. of the ACM SIGSOFT 20th Int. Symposium on the Foundations of Software Engineering*, page 27. ACM.

[19] M. Schultz, M. Eskin, E. Z. and Stolfo, F. (2001). Data mining methods for detection of new malicious executables. In *In Proc. of the 22nd IEEE Symposium on Security and Privacy*, page 38–49. IEEE.

[20] Matthew G. Schultz, E. E. and Zadok, F. (2001). Data mining methods for detection of new malicious executables. In *In Proc. of the 22nd IEEE Symposium on Security and Privacy*, page 38–49. IEEE.

[21] Moser, A., Kruegel, C., and Kirda, E. (2007). Exploring multiple execution paths for malware analysis. In *Security and Privacy, 2007. SP'07. IEEE Symposium on*, pages 231–245. IEEE.

[22] Park, Y., Reeves, D., Mulukutla, V., and Sundaravel, B. (2010). Fast malware classification by automated behavioral graph matching. In *Proceedings of the Sixth Annual Workshop on Cyber Security and Information Intelligence Research*, page 45. ACM.

[23] R. Islam, R. Tian, L. M. B. and Versteeg., S. (2013). Classification of malware based on integrated static and dynamic features. In *Journal of Network and Computer Applications*, page 36.

[24] R. Tian, L. M. B. and Versteeg, S. C. (2008). Function length as a tool for malware classification. In *In Proc. of the 3rd Int. Conf. on Malicious and Unwanted Software*, page 69–76. IEEE.

[25] S. Stolfo, K. W. and Li, W.-J. (2007). Towards stealthy malware detection. In *In Malware Detection*, page 231–249. Springer.

[26] Shankarapani, M., Kancherla, K., Ramammoorthy, S., Movva, R., and Mukkamala, S. (2010). Kernel machines for malware classification and similarity analysis. In *Neural Networks (IJCNN), The 2010 International Joint Conference on*, pages 1–6. IEEE.

[27] Tian R, Batten L, I. R. V. S. (2009). An automated classification system based on the strings of trojan and virus families. In *In Proceedings of the 4th international conference on malicious and unwanted software: MALWARE 2009*, pages 23–30.

[28] U. Bayer, P. M. Comparetti, C. H. C. K. and Kirda., E. (2009). Scalable, behavior-based malware clustering. In *In Proc. of the 16th Symposium on Network and Distributed System Security*.

[29] Wagener G, State R, D. A. (2008). Malware behaviour analysis. journal in computer virology. In *In Proceedings of the 5th International Conference on Malicious and Unwanted Software 2010*, pages 25–30. IEEE.

[30] Y. Ye, T. Li, Y. C. and Jiang, Q. (2010). Automatic malware categorization using cluster ensemble. In *In Proc. of the 16th ACM Int. Conf. on Knowledge Discovery and Data Mining*, page 95–104. ACM.

[31] Youngjoon Ki, E. K. and Kim, H. K. (2015). A novel approach to detect malware based on api call sequence analysis. *International Journal of Distributed Sensor Networks*, page 9.